C# Ders - 19 Ekim 2026 Ders Notları

1. Koşullu Döngüler: `while` vs `do-while` Farkı

Her iki döngü de bir koşula bağlı olarak çalışır; ancak **koşulun ilk kontrol edilme anı** çalışma mantığını tamamen değiştirir:

C#

```
bool status = false;

// 1. while Döngüsü
while (status)
{
    Console.WriteLine("burda dünyanın sırrını paylaşıyorum...");
}

// 2. do-while Döngüsü
do
{
    Console.WriteLine("burda dünyanın sırrını paylaşıyorum...");
}
while (status);
```

Karşılaştırma Analizi

Özellik	`while` Döngüsü	`do-while` Döngüsü
Kontrol Noktası	Döngünün **başında** (Girişte)	Döngünün **sonunda** (Çıkışta)
Minimum Çalışma Sayısı	`0` (Koşul baştan `false` ise gövdeye hiç girilmez)	`1` (Koşul `false` olsa dahi en az 1 kez çalışır)
Kullanım Alanı	Şart baştan sağlanmıyorsa kesinlikle çalışmaması gereken durumlar	Kullanıcıdan en az bir kez şifre/veri alıp sonra kontrol etme senaryoları

> **Kodun Çıktısı:** Yukarıdaki örnekte `status = false` olduğu için `while` bloğu ekrana hiçbir şey yazmaz; ancak `do-while` bloğu koşula sonda baktığı için ekrana **1 kez** *"burda dünyanın sırrını paylaşıyorum..."*yazdırır.

2. Klasik Algoritmik Problemler ve Çözüm Mantıkları

A. Asal Sayı Kontrolü ve Bayrak (Flag) Mantığı

Asal Sayı: 1'den ve kendisinden başka hiçbir pozitif tam sayıya tam bölünmeyen, 1'den büyük doğal sayılardır (2,3,5,7,11...2,3,5,7,11...).

Algoritmada **Bayrak (Flag)** tekniği kullanılır: Sayı baştan asal kabul edilir (`isAsal = true`). Sayıyı 22'den başlayarak kendisine kadar olan sayılara böleriz:

C#

```
Console.WriteLine("ASAL Kontrolü yapılacak sayıyı giriniz:");
int.TryParse(Console.ReadLine(), out int n);

if (n > 2)
{
    bool isAsal = true;
    int divide = 2;

    while (divide < n)
    {
        if (n % divide == 0) // Eğer tam bölünüyorsa asal olamaz!
        {
            isAsal = false;
            break; // Daha fazla bölmeye gerek yok, döngüyü hemen kır!
        }
        divide++;
    }

    string durum = isAsal ? $"{n} ASAL bir sayıdır" : $"{n} ASAL bir sayı değildir";
    Console.WriteLine(durum);
}
```

- `break;` **Önemi:** Bir bölen bulunduğu an sayının asal olmadığı kesinleşir. `break` ifadesi döngüyü erken keserek gereksiz CPU turlarını engeller (Algoritmik Verimlilik).
- *(Ek Matematiksel İpucu: Bir sayının bölenlerini ararken nn'e kadar gitmek yerine nn'e kadar (`Math.Sqrt(n)`) gitmek algoritmayı binlerce kat hızlandırır).*

B. Belirli Bir Sayıya Kadarki Asal Sayıları Listeleme (`StringBuilder` ile)

İç içe iki döngü kullanılır:

1. **Dış Döngü (`i`):** 33'ten başlayarak hedef sayıya kadar tek tek ilerler.
2. **İç Döngü (`j`):** O anki `i` sayısının asal olup olmadığını test eder.
3. `StringBuilder`**:** Asal bulunan sayıları string birleştirme (`+=`) maliyetine girmeden bellekte optimize bir şekilde toplar.

C#

```
StringBuilder sb = new StringBuilder("1, 2"); // 1 ve 2 başlangıç olarak eklenmiş

for (int i = 3; i < n; i++)
{
    bool isAsal = true;
    for (int j = 2; j < i; j++)
    {
        if (i % j == 0)
        {
            isAsal = false;
            break;
        }
    }

    if (isAsal)
    {
        sb.Append($", {i}");
    }
}
Console.WriteLine(sb.ToString());
```

C. Faktöriyel Hesaplama

$n! = n \times (n-1) \times (n-2) \ldots \times 1$

Sayılar çok hızlı büyüdüğü için `int` yerine `long` veri tipi tercih edilir:

C#

```
Console.WriteLine("Faktöriyeli hesaplanacak sayıyı giriniz:");
long.TryParse(Console.ReadLine(), out long n);

long faktoryel = n;
for (long i = n - 1; i > 0; i--)
{
    faktoryel *= i; // Kümülatif çarpım
```

```
}
Console.WriteLine($"{n} sayısının faktöriyeli: {faktoryel}");
```

D. Fibonacci Serisi ve Değişken Takası (Swapping)

Seri: 0,1,1,2,3,5,8,13,21...0,1,1,2,3,5,8,13,21... (Her sayı kendinden önceki iki sayının toplamıdır).

Burada 3 değişkenli kaydırma mekanizması kullanılır:

- aa: Mevcut yazdırılacak sayı
- bb: Bir sonraki sayı
- temptemp: aa'nın eski değerini kaybetmemek için geçici tutucu

C#

```
if (int.TryParse(Console.ReadLine(), out int n) && n > 0)
{
    long a = 0;
    long b = 1;

    for (int i = 0; i < n; i++)
    {
        Console.Write(a + " ");

        long temp = a; // a'nın değerini yedekle
        a = b;         // Yeni terim, eski b oldu
        b = temp + b;  // Yeni b, eski a ile b'nin toplamı oldu
    }
}
```

3. İç İçe Döngüler (Nested Loops) ve 2B Koordinat Düzlemi

Konsolda şekil veya matris çizerken:

- **Dış Döngü (`i`): Satırları (Y ekseni)** temsil eder. Dış döngünün her turunda `Console.WriteLine();` ile alt satıra geçilir.
- **İç Döngü (`j`): Sütunları (X ekseni)** temsil eder. Yan yana karakter basmak için `Console.Write();` kullanılır.

A. İçi Boş Kare Çizimi

Bir karenin sınırları: Üst kenar (`i == 0`), alt kenar (`i == n`), sol kenar (`j == 0`) ve sağ kenardır (`j == n`).

C#

```
int n = 5;
for (int i = 0; i <= n; i++) // Satırlar
{
    for (int j = 0; j <= n; j++) // Sütunlar
    {
        // Yalnızca kenarlarda yıldız bas, iç kısımlara boşluk bırak:
        if (i == 0 || j == 0 || i == n || j == n)
        {
            Console.Write("*");
        }
        else
        {
            Console.Write(" ");
        }
    }
    Console.WriteLine(); // Satır bittiğinde alt satıra geç
}
```

Plaintext

```
Çıktı:
******
*    *
*    *
*    *
*    *
******
```

B. İçi Boş Dik Üçgen Çizimi

Üçgenin sınırları: Sol kenar (`j == 0`), taban kenarı (`i == n`) ve hipotenüs/çaprazdır (`i == j`).

C#

```
int n = 10;
for (int i = 0; i <= n; i++)
```

```
{
    for (int j = 0; j <= i; j++) // Sütun sadece satır numarasına kadar gider
    {
        if (i == 0 || i == j || i == n || j == 0)
        {
            Console.Write("*");
        }
        else
        {
            Console.Write(" ");
        }
    }
    Console.WriteLine();
}
```

C. Simetrik Çam Ağacı / Piramit Çizimi

Piramit oluştururken iki aşama vardır: Önce satır başına **boşluk**, ardından tek sayıda (2i−12i−1) **yıldız** basılır:

C#

```
int yuk = 10;

for (int i = 1; i <= yuk; i++)
{
    // 1. Boşluk basan döngü (yuk - i adet)
    for (int j = 1; j <= yuk - i; j++)
    {
        Console.Write(" ");
    }

    // 2. Yıldız basan döngü (2*i - 1 adet tek sayı formülü: 1, 3, 5, 7...)
    for (int k = 1; k <= (2 * i - 1); k++)
    {
        Console.Write("*");
    }

    Console.WriteLine();
}
```

4. Zaman Yönetimi ve Gecikme: `Thread.Sleep()`

Kodundaki "Kapıcı İbo" örneğinde geçen `Thread.Sleep(2000);` ifadesi:

- Programın çalıştığı mevcut iş parçacığını (thread) belirtilen milisaniye kadar **uyutur / duraklatır**.
- `1000 ms = 1 saniye` olduğundan, `Thread.Sleep(2000);` programı **2 saniye boyunca dondurur**.
- Konsol oyunlarında, animasyonlarda, adım adım ilerleyen simülasyonlarda veya API istek limitlerini aşmamak için kullanılır.

C#

```
int kat = 3, daire = 4;
int kapiNo = 1;

for (int i = 1; i <= kat; i++)
{
    Console.WriteLine($"--- {i}. Kata Gelindi ---");
    for (int j = 1; j <= daire; j++)
    {
        Console.WriteLine($"{i}. Kat, {j}. Daire (Kapı No: {kapiNo}) zili çalınıyor...");
        kapiNo++;
        Thread.Sleep(1000); // 1 saniye bekle
    }
}
```

5. Koleksiyon Gezinimi: `for` vs `foreach`

Dizilerde ve listelerde elemanları dolaşmak için iki temel döngü kullanılır:

C#

```
string[] names = { "Abdullah", "Ayşe", "Kezban", "Şukufe", "Murtaza", "Musa" };

// 1. Klasik for Döngüsü
for (int i = 0; i < names.Length; i++)
{
    Console.WriteLine(names[i]); // İndeks ile erişim
}

// 2. foreach Döngüsü
foreach (string name in names)
{
    Console.WriteLine(name); // Doğrudan elemanın kendisi gelir
}
```

`for` ile `foreach` Arasındaki Farklar

Kriter	`for` Döngüsü	`foreach` Döngüsü
Çalışma Şekli	Bir sayaç (`i`) üzerinden dizi indekslerini tek tek gezer.	Arka planda `IEnumerator` yapısını kullanarak elemanları sırayla çeker.
İndeks Bilgisi	`i` değişkeni sayesinde elemanın kaçıncı sırada olduğu bilinir.	Doğrudan bir indeks bilgisi yoktur, sadece eleman vardır.
Eleman Değiştirme	Dizi elemanı güncellenebilir (`names[i] = "Yeni";`).	**Salt Okunurdur (Read-Only)!** Döngü içinde eleman değiştirilemez (`name = "X";` derleme hatası verir).
Yön ve Adım	Geriye doğru (`i--`) veya ikişer ikişer (`i += 2`) atlanabilir.	Sadece baştan sona, tek yönlü ve teker teker ilerler.
Okunabilirlik	Daha fazla kod ve sayaç yönetimi gerektirir.	Çok daha temiz, hatasız ve okunabilirdir.